

REST Web Services

Desarrollo de Software en Sistemas Distribuidos

REST: historia

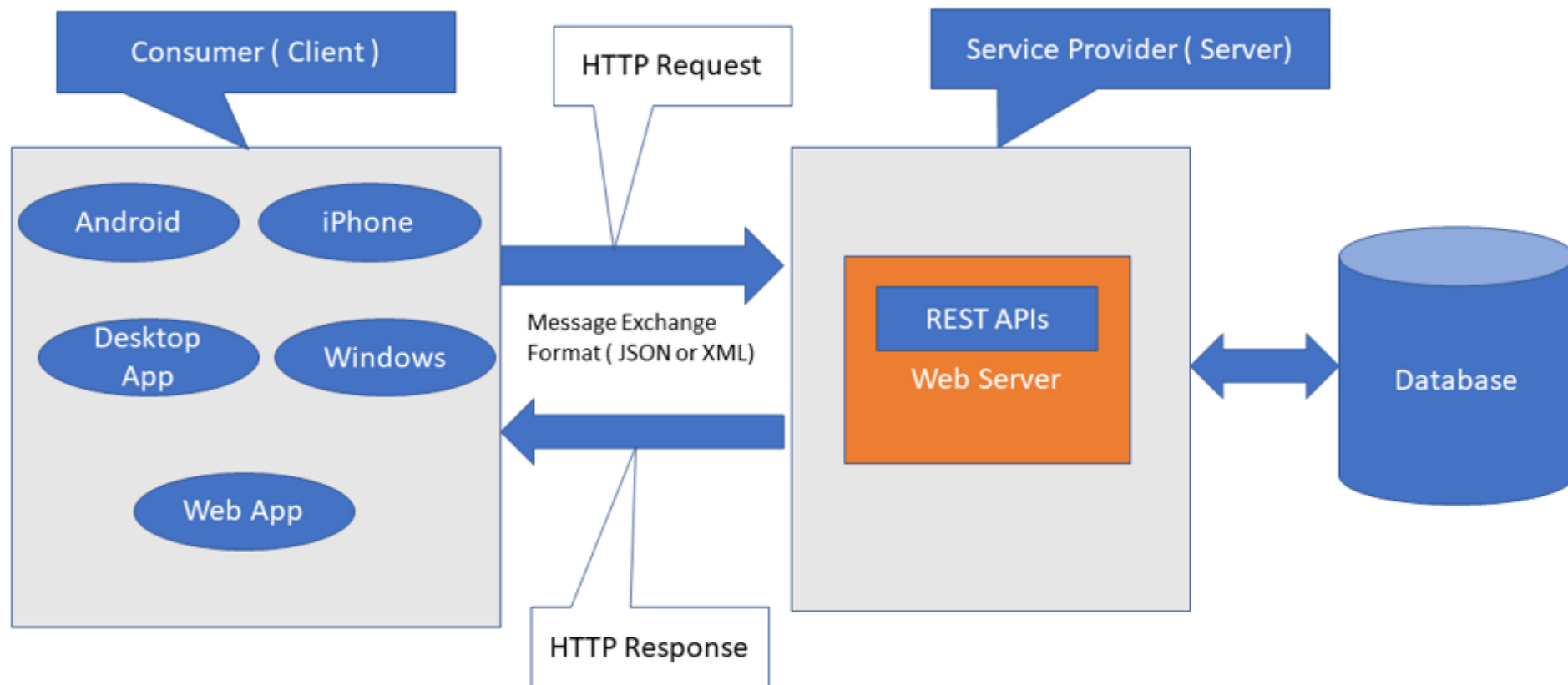
- ▶ Roy Fielding: el cráneo detrás de REST
 - ▶ Participó en el desarrollo de HTML
 - ▶ Fue uno de los fundadores de Apache
 - ▶ En el año 2000 publica su disertación titulada *“Architectural Styles and the Design of Network-based Software Architectures”* (Estilos Arquitectónicos y Diseño de Arquitecturas de Software basadas en Redes) donde expone la arquitectura REST en el capítulo 5
 - ▶ https://www.ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf



REST: Características

REpresentational State Transfer (Transferencia de estado representacional), es un estilo arquitectónico que establece pautas para el diseño y desarrollo de servicios en internet.

REST – Architecture



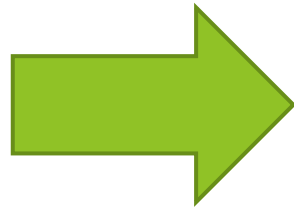
REST: Reglas/Principios/Restricciones

- ▶ Las reglas que conforman la arquitectura REST son:
 - ▶ Interfaz uniforme
 - ▶ Separación Cliente-Servidor
 - ▶ Peticiones sin estado
 - ▶ Cacheable
 - ▶ Sistema por capas
 - ▶ Código a demanda (opcional)

REST: Reglas/Principios/Restricciones

► Interfaz uniforme

- Clara
- Uniforme
- Consistente
- Comprensible



- ✓ *Identificación de recursos por URLs*
- ✓ *Mensajes en formato estándar*
- ✓ *Acciones mediante métodos HTTP*

REST: Reglas/Principios/Restricciones

- ▶ **Separación Cliente-Servidor**
 - ▶ Independientes
 - ▶ Evolucionan por separado
 - ▶ Servidor centrado en almacenamiento y recursos
 - ▶ Cliente enfocado en presentación e interacción con el usuario

REST: Reglas/Principios/Restricciones

- ▶ **Peticiones sin estado (Stateless)**

- ▶ **Servidor:**

- ▶ No existen sesiones
 - ▶ Cada solicitud es independiente

- ▶ **Cliente:**

- ▶ Debe enviar toda la información necesaria
 - ▶ Debe manejar estados previos si los hubiere

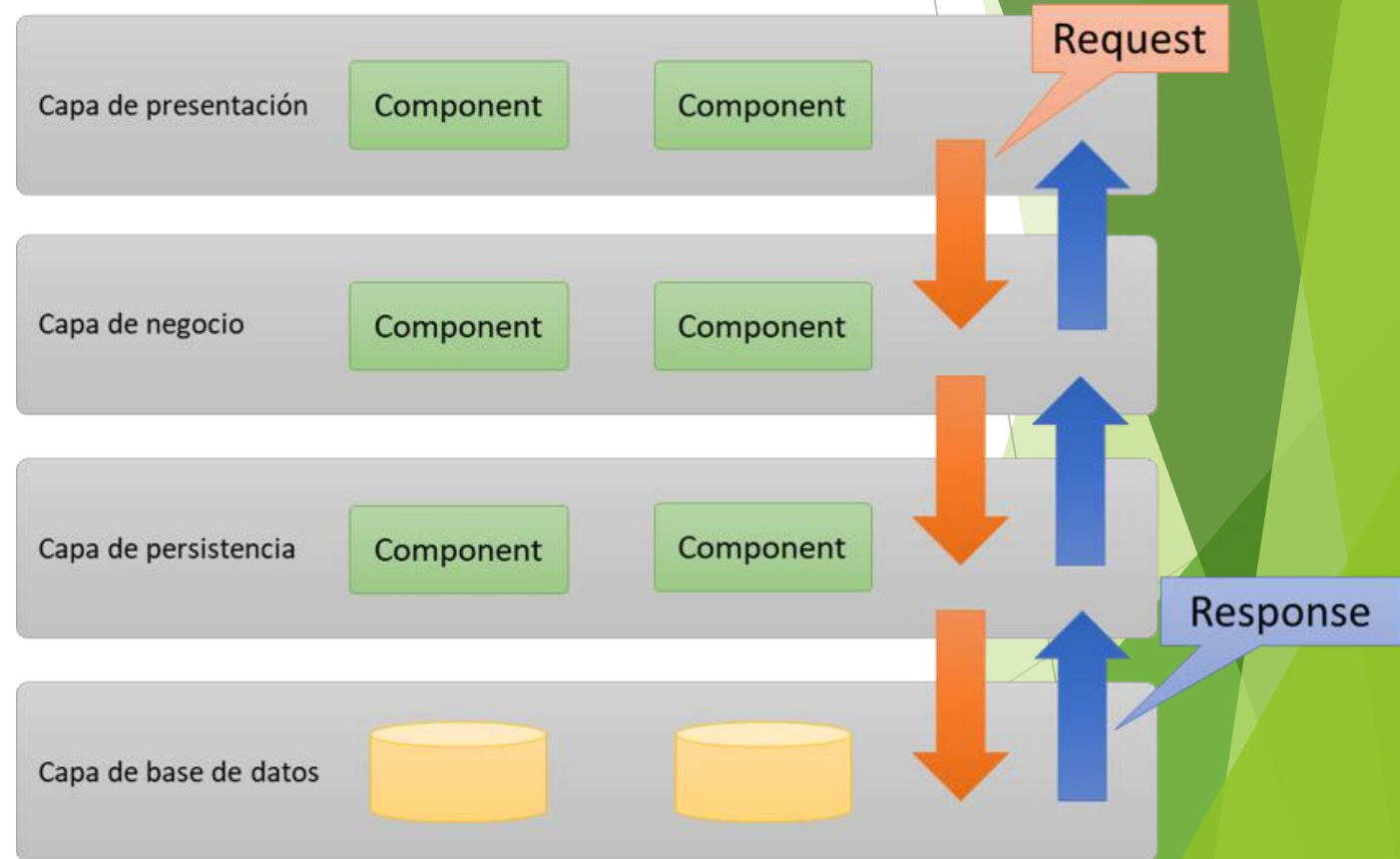
REST: Reglas/Principios/Restricciones

- ▶ **Cacheable**
 - ▶ Reducir la carga y uso de red
 - ▶ Servidor
 - ▶ Cliente
 - ▶ Intermediarios (api gateway)

REST: Reglas/Principios/Restricciones

► Sistema por capas

- Separación de responsabilidades
- Flexibilidad
- Escalabilidad
- Mantenibilidad
- Seguridad
- Reusabilidad



REST: Reglas/Principios/Restricciones

- ▶ **Código a demanda (opcional)**
 - ▶ **Servidor puede enviar código ejecutable**
 - ▶ Javascript
 - ▶ Applets
 - ▶ Etc
 - ▶ **Extiende las funcionalidades del cliente**
 - ▶ Formularios interactivos
 - ▶ Validaciones
 - ▶ Actualizaciones
 - ▶ Etc

REST: Elementos

- ▶ **Recurso:** documento, archivo, entidad de negocio, etc...
- ▶ **Identificador del recurso:** <http://example.com/employees/1>
- ▶ **Representación:** JSON, XML, HTML, etc...
- ▶ **Datos de control:** Acciones (Verbos) HTTP y respuestas (200, 404, 500, etc...)

REST/RESTFul API: Definición

- ▶ Permite la interacción con los servicios web basados en los principios de la arquitectura REST.
- ▶ Utiliza el protocolo HTTP para la comunicación
- ▶ Representaciones en formato JSON, XML, Texto...
- ▶ Los recursos se identifican mediante una URL. Por ejemplo, si quisiéramos consultar los datos de un alumno, apuntaríamos: <http://www.unla.edu.ar/restapi/alumnos/12345> y nos devolvería su representación:

```
{  
  "nombre": "Marcos Amaro",  
  "carrera": "Sistemas",  
  "legajo": 12345,  
  ...  
}
```

- ▶ Maneja diferentes códigos de respuesta, ejemplo: 404 si no existe el recurso.
- ▶ https://es.wikipedia.org/wiki/Anexo:C%C3%B3digos_de_estado_HTTP

REST: Operaciones

- ▶ Basadas en el principio CRUD:
 - ▶ Create (Crear)
 - ▶ Retrieve (Obtener)
 - ▶ Update (Actualizar)
 - ▶ Delete (Borrar)
- ▶ <https://developer.mozilla.org/es/docs/Web/HTTP/Methods>

REST: Operaciones

▶ POST

- ▶ Utilizado para la creación de un recurso.
- ▶ Los datos se envían en el cuerpo de la solicitud (body)
- ▶ Respuesta 201 (Created)

REST: Operaciones

▶ GET

- ▶ Recuperación/obtención de recursos.
- ▶ No debe alterar recursos.
- ▶ **Datos en la URL (Query String)**
- ▶ Puede ser cacheable
- ▶ Respuesta 200 (OK) o 404 (NOT FOUND)
- ▶ *Se propuso un método llamado QUERY*

REST: Operaciones

▶ PUT

- ▶ Actualización completa de recursos.
- ▶ Se debe enviar la representación completa.
- ▶ Puede crear el recurso si no existe.
- ▶ Respuesta 200 (OK) o 201 (Created)
- ▶ *Alternativa PATCH para actualización parcial*

REST: Operaciones

▶ DELETE

- ▶ Eliminación de recursos.
- ▶ Se envía el identificador por URL.
- ▶ Puede ser borrado lógico o físico.
- ▶ Respuesta 204 (No content), 200 (OK), 202 (Accepted)

REST: Estandarización de respuestas

► Especificación JSend

```
{  
  "status": "fail",  
  "message": "Error de validación",  
  "data": {  
    "email": "El email ya está registrado",  
    "password": "Debe tener al menos 8 caracteres"  
  }  
}
```

```
{  
  "status": "success",  
  "data": {  
    "empleado": {  
      "id": 123,  
      "nombre": "Juan Pérez"  
    }  
  }  
}
```

Ejemplo

Backend: <https://github.com/JulianTomczak/api-rest-dssd>

Frontend: <https://github.com/JulianTomczak/front-apirest>

Documentación/Test con Swagger

- ▶ Swagger es un conjunto de herramientas de software de código abierto para diseñar, construir, documentar, y utilizar servicios web RESTful.
- ▶ Contiene un editor online <https://editor.swagger.io/> o bien se puede descargarlo para trabajar offline <https://github.com/swagger-api/swagger-editor/releases> o Docker:
`docker run -d -p 8090:8080 docker.swagger.io/swaggerapi/swagger-editor`
- ▶ Se puede integrar a proyectos y generar documentación directamente sobre el código fuente de las REST Api.
- ▶ En este ejemplo se utiliza el editor generando la documentación manualmente con la especificación de openapi 3.0
- ▶ Para mayores detalles, consultar la documentación oficial:
<https://swagger.io/docs/specification/basic-structure/>

Documentación/Test con Swagger

- ▶ En primer lugar se informa la especificación utilizada y su versión.
- ▶ Luego la sección de info con detalles sobre la documentación.
- ▶ En servers se pueden especificar diferentes urls donde se puede llamar a la api, por ejemplo sitio de test, producción, etc. Es una sección de tipo array, cada elemento del array se inicia con guión -

```
openapi: "3.0.0"
info:
  title: Cursos de Programación EdUNLa
  description: Rest API de EdUNLa para el manejo de cursos de programación
  version: "1.0"

servers:
  - url: http://127.0.0.1:5000
    description: "URL de la Rest API"
```

Documentación/Test con Swagger

- ▶ La sección de paths contendrá las rutas expuestas por la api con la operación que acepta cada una (get, post, etc).
- ▶ Cada operación genera una respuesta con su respectivo código.
- ▶ Los paths pueden recibir parámetros en la url, se pueden definir a nivel global por path o dentro de cada operación.
- ▶ La propiedad \$ref indica que se utiliza una referencia especificada en otra parte del documento.

```
paths:
  /cursos:
    get:
      tags:
        - Curso
      summary: Devuelve todos los cursos disponibles
      responses:
        "200":
          description: OK
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/ArrayOfCurso"

    post:

  /cursos/{id}:
    parameters:
      - name: id
        in: path
        required: true
        description: Identificador del curso a consultar
        schema:
          type: integer
          example: 1
    get:
      tags:
        - Curso
      summary: Devuelve el curso especificado
      responses:
        "404":
          $ref: "#/components/responses/NotFound"
        "200":
          $ref: "#/components/responses/Successful"
```

Documentación/Test con Swagger

- ▶ En esta sección se especifican las entidades que pueden ser referenciadas en otras partes con la propiedad \$ref
- ▶ Cada tipo especificado contiene propiedades, también se puede indicar cuáles son requeridas y poner un ejemplo de su representación tanto a nivel global como a nivel de cada propiedad.

```
components:
  schemas:
    Curso:
      type: object
      required:
        - nombre
        - precio
        - duracion
      properties:
        id:
          type: integer
          description: Identificador numérico del curso
        lenguaje:
          type: string
          description: Nombre del curso
        precio:
          type: number
          description: Precio total del curso
        duracion:
          type: integer
          description: La duración del curso en meses
      example:
        id: 1000
        lenguaje: Java
        precio: 449.95
        duracion: 4
```

Documentación/Test con Swagger

- En la parte derecha se van creando cajas en las cuales se puede ver de forma más amigable las operaciones disponibles con los ejemplos y la posibilidad de ejecutar cada una de las acciones.

Curso ▼

- GET** `/cursos` Devuelve todos los cursos disponibles
- POST** `/cursos` Crea un nuevo curso
- GET** `/cursos/{id}` Devuelve el curso especificado ↩
- PUT** `/cursos/{id}` Modifica un curso existente
- DELETE** `/cursos/{id}` Elimina un curso existente

Schemas >

Documentación/Test con Swagger

GET /cursos/{id} Devuelve el curso especificado

Parameters Cancel

Name	Description
id * required integer (path)	Identificador del curso a consultar
3	

Execute Clear

Curl

```
curl -X 'GET' \  
'http://127.0.0.1:5000/cursos/3' \  
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:5000/cursos/3
```

Server response

Code	Details
200	Response body <pre>{ "duracion": 6, "id": 3, "lenguaje": "Erlang", "precio": 5000 }</pre> Download

Response headers